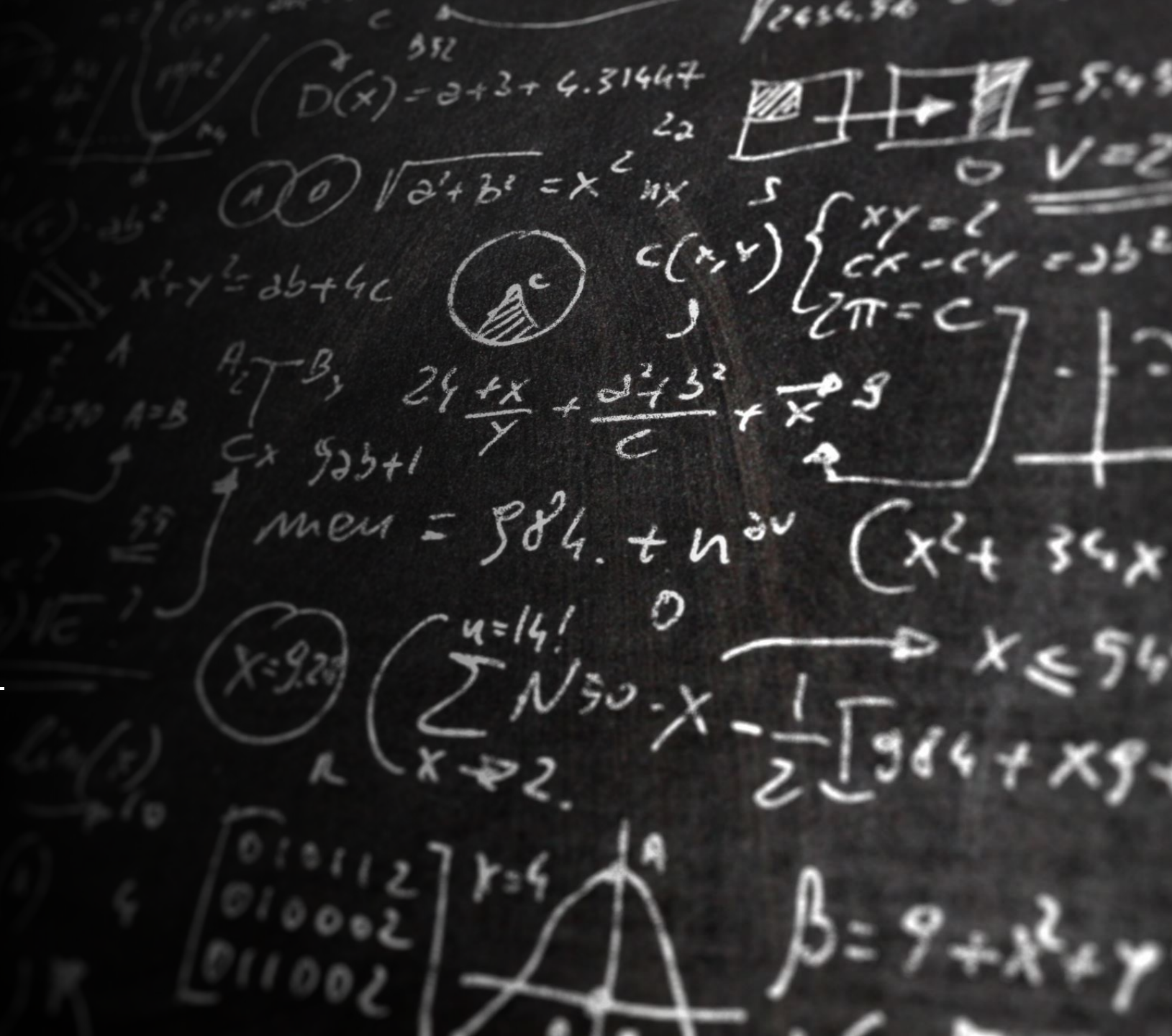
Complexity Theory

Malav Patel

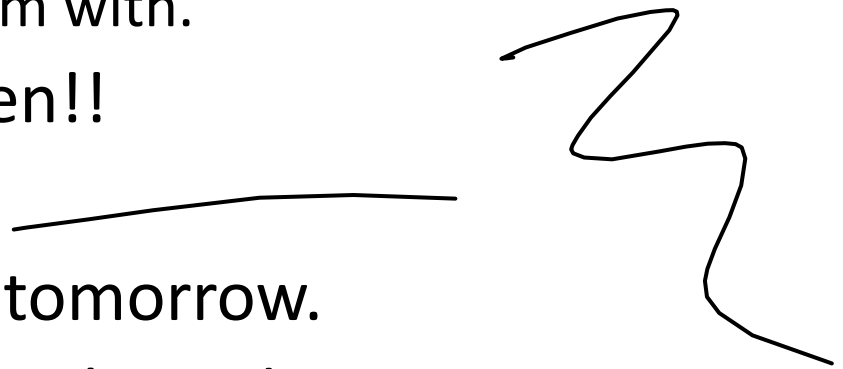
Himanshu Goyal

Rohit A.



Announcements

- Study Guide has been posted on Canvas.
- Exam 4 (in-person and on paper) tomorrow (04/18)
  - Please bring something to write the exam with.
- Apply to be a TA. Applications are open!!
- CIOS Bonus will be out…
- We'll release a excel grade calculator tomorrow.
- Solutions to Practice Exam have been released.
- The optional assignment (PS9) will be due until April 23th and late due April 24th. ☺
- Final Exam is May 4th.



Topics

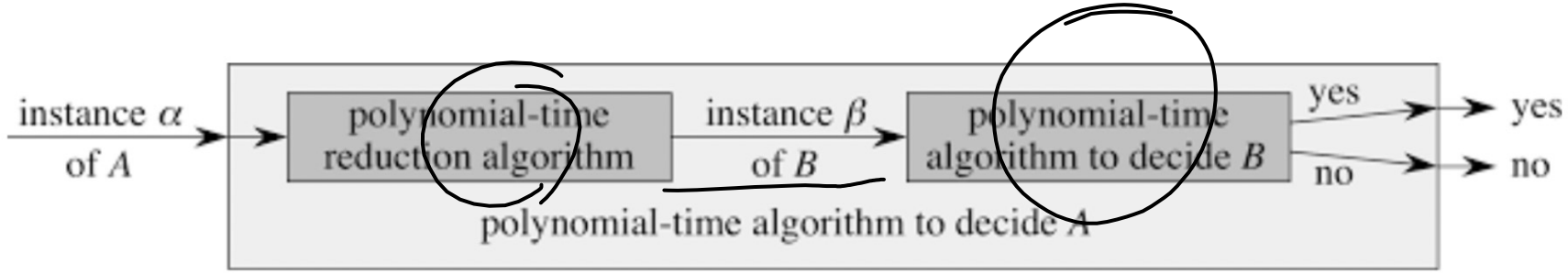
Check out the study guide and practice sheets!

- Intro to Complexity
- SAT, 3-SAT
- Independent Set, Clique, Vertex Cover
- Knapsack Problem
- Subset-Sum
- 3-Coloring
- Practice Exam 4

Definitions

- NP: Nondeterministic Polynomial. Can you VERIFY the problem in polynomial time?
  - Verify: Given a solution to a problem, can we verify that solution in polynomial time?
    - Ex.
      - Problem: Find 3 terms (a,b,c) from list L where a+b+c = k where k is a specific integer
      - Input: List L, int k
      - Output: {a,b,c} where a+b+c = k
        - We can verify that a,b, and c are in list L and that a+b+c=k

Reductions



Problem of type A and an algorithm of type B

- Convert the type A problem into type B

If there exists a POLYTIME reduction from A to B where A is a KNOWN NP-Complete/NP-Hard problem, then B is NP hard.

Ex. You are trying to get melted cheese but you don't know how to do that.
You do, however, know someone who can make pizza. From a pizza, you can easily get melted cheese (in polynomial time)

Intuition Behind Why a Reduction works?

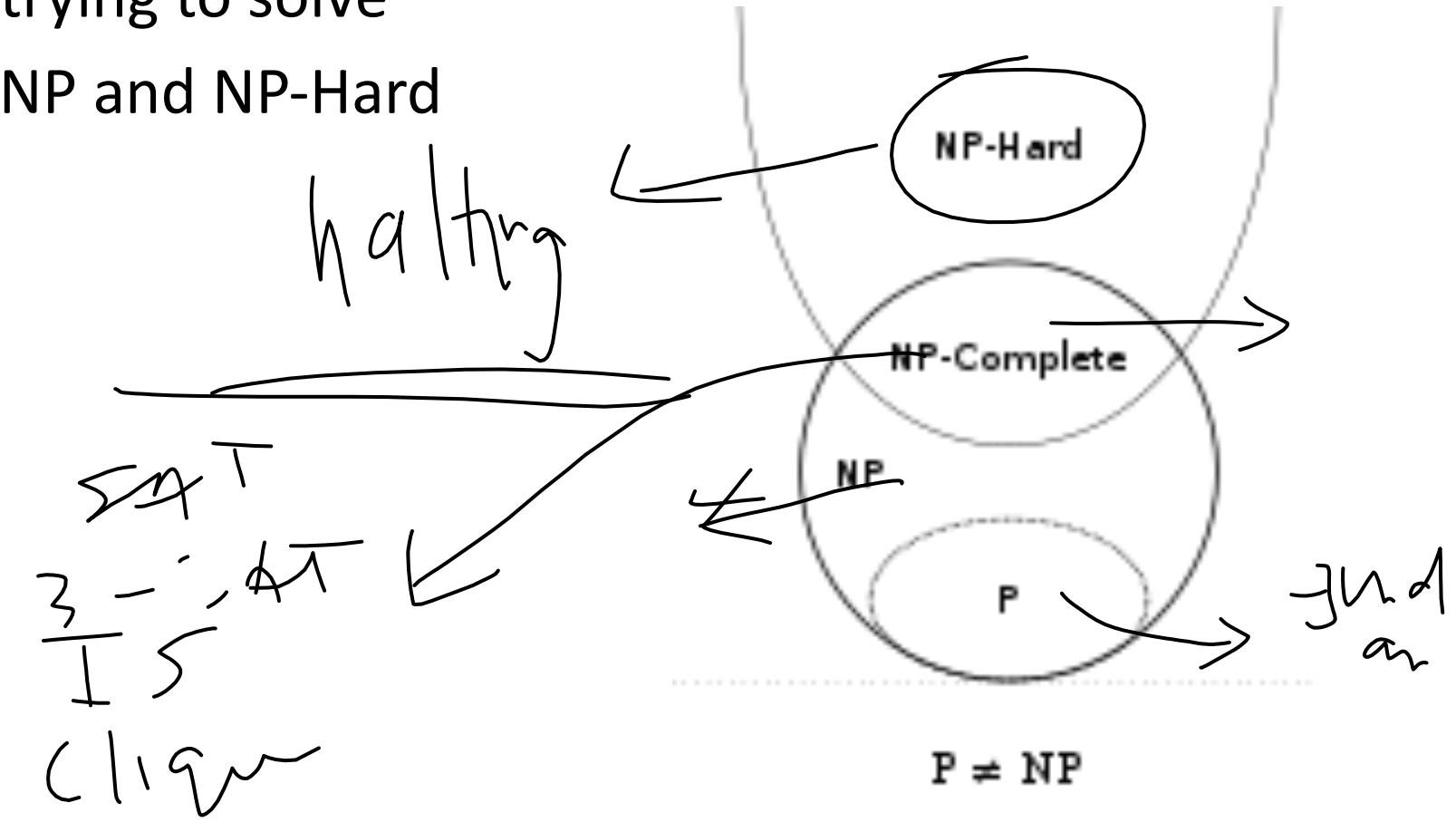
- After taking CS 3510, you become a PhD student
- Your advisor has asked you to solve the problem of whether to put pineapple on a pizza or not given any random pizza (Pineapple)
- You've spent an year trying to figure out an answer to the question and you suspect it's too hard a problem to solve efficiently
- **Can I prove that this problem is NP-Hard? Then you won't get crucified by either faction**

Reducing from 3-SAT to Pineapple?

- I need to use Pineapple to solve 3SAT
  - Why? - Use a proof by contradiction
  - Assume that Pineapple is NOT NP-Hard
  - If I can use Pineapple to solve 3SAT in polytime (after the mappings)
  - I have solved 3SAT in polytime – which is a contradiction as we know it is NP-Hard (caveats here though)
  - Therefore, Pineapple must also be NP-Hard
- We use the unknown (Pineapple) to solve the known (3SAT) to arrive at conclusions about the unknown (Pineapple)
- Therefore, too hard to solve for pineapple (your advisor and you are happy!)

Definitions

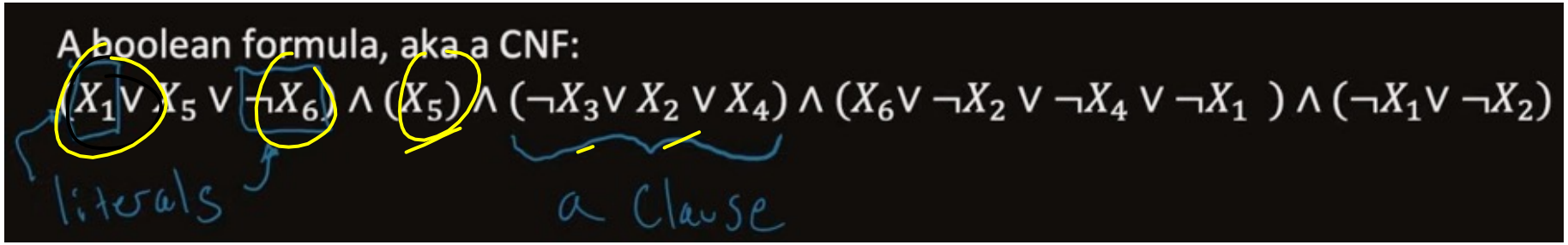
- NP-Hard: Being able to reduce from an already NP-Hard problem to the problem you are trying to solve
- NP-complete= BOTH NP and NP-Hard



How to make a Million Dollars

- Solve an NP-Hard problem in poly time
  - P = NP
- Show that there are problems that you can verify in polynomial time but not solve in polynomial time.
  - P != NP
  - This would need to be a proof about generic transformations of problems as it is difficult to prove that you have the best algorithm possible
- If you get either one of these, DM the Clay Mathematics Institute and they'll venmo the money (https://www.claymath.org/)

SAT



SAT = a CNF formula with any number of literals per clause

Inputs: CNF, literals with assignments (L)
Output: True/False (is L a valid assignment)

1. SAT is NP:
Given an assignment we can easily prove that the Boolean formula is true

2. SAT is NP-Complete:

Proved using Cook-Levin Theorem

3-SAT

$$\left(\overline{x_1} \vee x_2 \vee x_3\right) \wedge \left(x_1 \vee \overline{x_2} \vee x_3\right) \wedge \left(x_1 \vee x_2 \vee x_4\right) \wedge \left(\overline{x_1} \vee \overline{x_3} \vee \overline{x_4}\right)$$

3-SAT = A CNF with

Inputs: CNF, literals with assignments (L)
Output: True/False (is L a valid assignment)

1. 3-SAT is NP:
Given an assignment we can easily prove that the Boolean formula is true

Ex. $(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3 \vee \neg x_4) \wedge (x_2 \vee x_4 \vee \neg x_5) \wedge (\neg x_3 \vee x_5)$

- To make this statement true, if we are given a solution $x_1 = F, x_2 = F, x_3 = T, x_4 = T$ and $x_5 = T$, it is easy to verify in polynomial time that the solution works for our given SAT problem

2. 3-SAT is NP-Complete:
Reducible from SAT

Example Problem

Input: A boolean function in CNF.
Output: Three distinct satisfying assignments that evaluate the given function as TRUE if such an assignment exists.

Prove that the 3Satisfying_SAT is NP-Complete.

Prove it is in NP:
3Satisfying_SAT is in NP. We can design a polynomial time algorithm in the following way. We are given three satisfying assignments (solutions). We can run an $O(literals)$ to compare the values of $X_i's$ across all three assignments. If all the values of any of the two assignments are the same, we can return False. Otherwise, we can test the values of each of the assignments in the given CNF. If the conjunction of the outputs of the three assignments is True, then we return True. In this way, we can verify if the given assignments are satisfying assignments for 3Satisfying_SAT. This algorithm works because it checks if the assignments are unique and if they satisfy the given CNF in $O(literals)$, which is a polynomial time.

Reduction:

Given an input to the SAT problem, we simply construct a new input to 3Satisfying_SAT by adding a clause $a \cup b$, where a and b do not belong to the input. For example, Let our input CNF to SAT be:
$(X_1 \cup X_2 \cup X_3) \cap (X_1 \cup X_2') \cap (X_3 \cup X_4')$
then our input to 3Satisfying_SAT will become,
$(X_1 \cup X_2 \cup X_3) \cap (X_1 \cup X_2') \cap (X_3 \cup X_4') \cap (a \cup b)$

Reduction Proof:

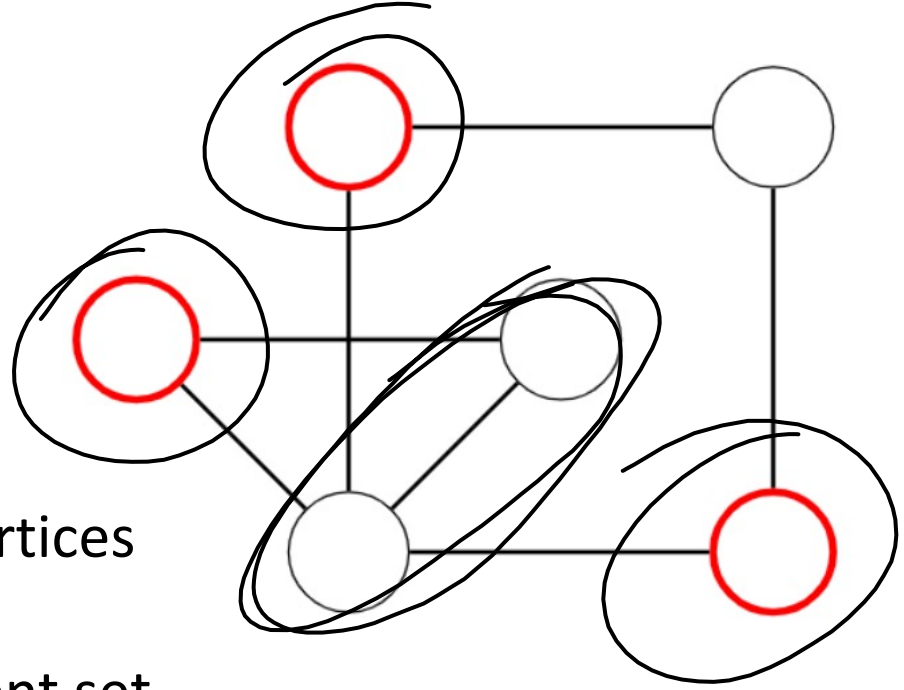
Claim: There are three distinct assignments for this new input to our 3Satisfying_SAT, if and only if there's an assignment for our original input for SAT. We'll show this correction in both directions:

(a) ($\rightarrow$) If our function 3Satisfying_SAT returns true with the new CNF, then we can guarantee that we have an assignment for SAT as well. This is because the CNF consists of $(a \cup b)$ which is true for exactly three pairs of inputs $(a = T, B = T), (a = T, B = F)$, and $(a = F, B = T)$. If there's an assignment for 3Satisfying_SAT, we can say that everything remains satisfied after removing this clause, and we have an assignment for SAT.

(b) ($\leftarrow$) If SAT has an assignment, then in that assignment $(a \cup b)$ must be true if $(a = T, B = T), (a = T, B = F)$, and $(a = F, B = T)$. If SAT has an assignment, then we can say that adding $(a \cup b)$ would create three distinct assignments with three different combinations of a and b. Hence, if we have an assignment for SAT, then adding this new clause would create three distinct with three different combinations. Therefore, we have three distinct satisfying assignments that evaluate 3Satisfying_SAT to be TRUE.

$x \in A \;,\; f(x) \in B$

Independent Set



Independent set = set of vertices where none of the vertices have connecting edges

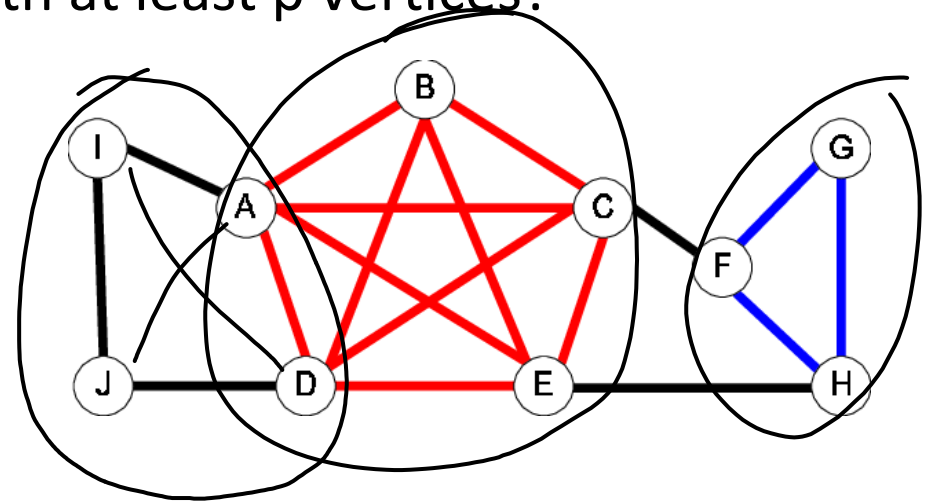
- Independent Set Problem: Is there an independent set in graph G with **at least p vertices**?
- **Theorem:** Independent set (IS) is NP-Complete

Input: Graph G, goal p (goal = Number of vertices)
Output: True/False (is there a set in graph G with AT LEAST p vertices)

Clique

- Clique = set of vertices where all vertices have edges to each other (everyone touches everyone – but not in a weird way)
  - Clique Problem: Is there a clique in graph G with at least p vertices?
  - **Theorem:** Clique is NP-Complete

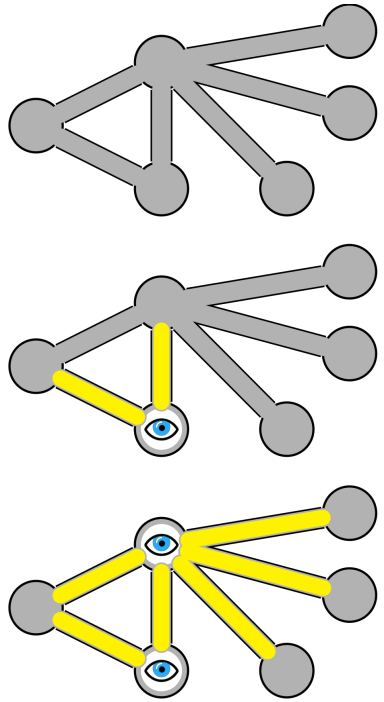
Inputs: Graph G, clique size k
Output: True/False (if clique of size k present in graph)



Vertex Cover

- Minimum Vertex Cover = set of vertices that covers at least one endpoint of every edge in a graph
  - Vertex Cover Problem: Is there a vertex cover in graph G with at most p vertices?
  - Theorem: Vertex Cover is NP-Complete

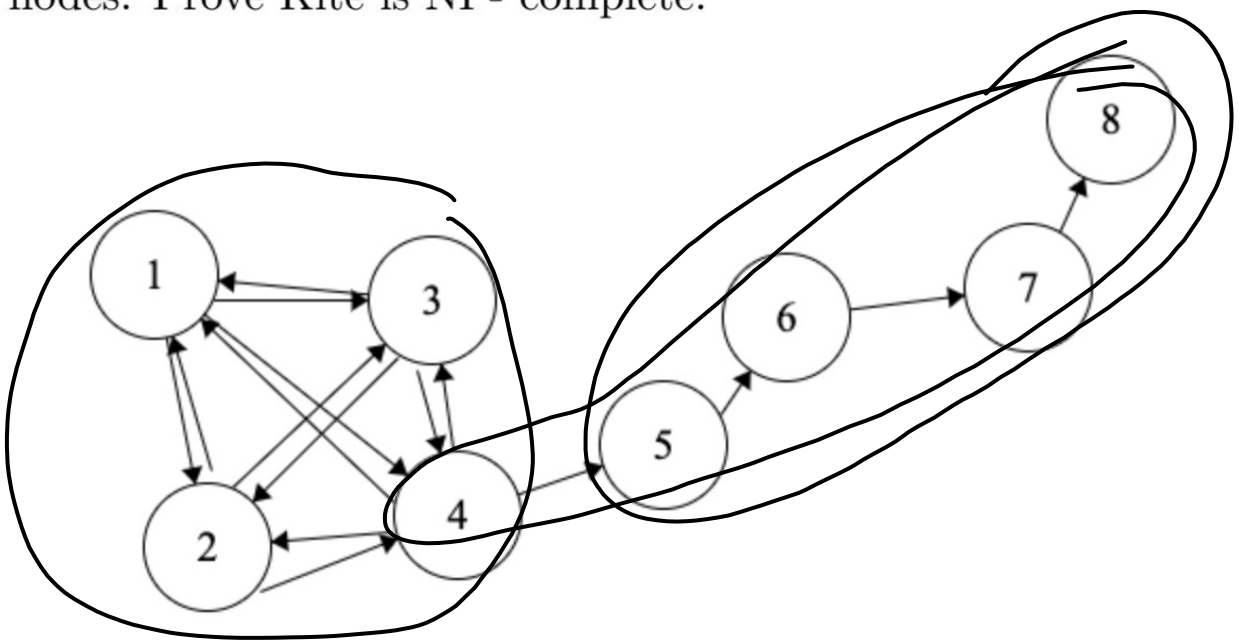
Inputs: Graph G, vertex cover size k
Output: True/False (if vertex cover of size k or less present for graph)



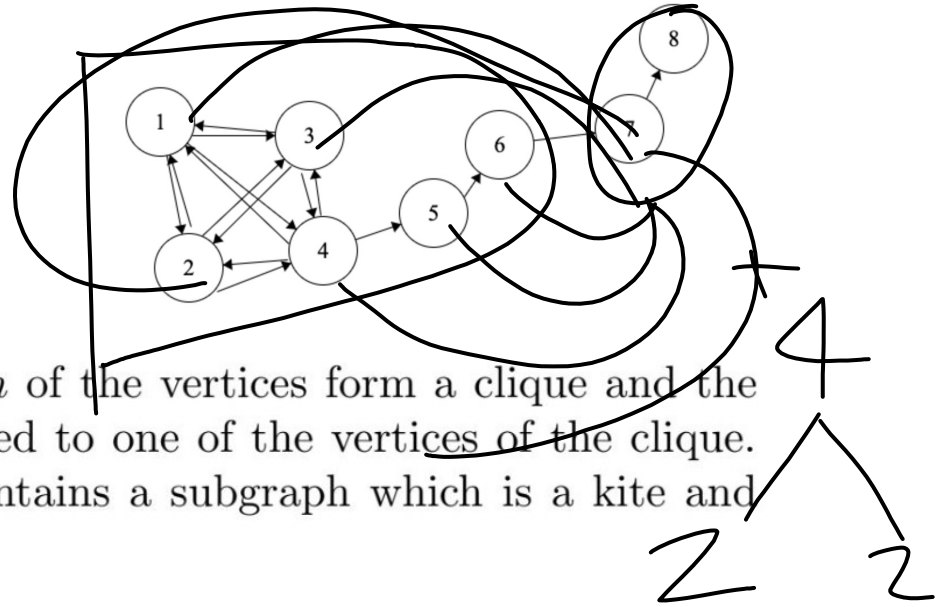
Example Problem

A kite is a graph on an even number of vertices, say $2n$, in which n of the vertices form a clique and the remaining n vertices are connected in a that consists of a path joined to one of the vertices of the clique. Given a graph and a goal g, the KITE problem asks whether G contains a subgraph which is a kite and which contains $2g$ nodes. Prove Kite is NP- complete.

N=4 2x4



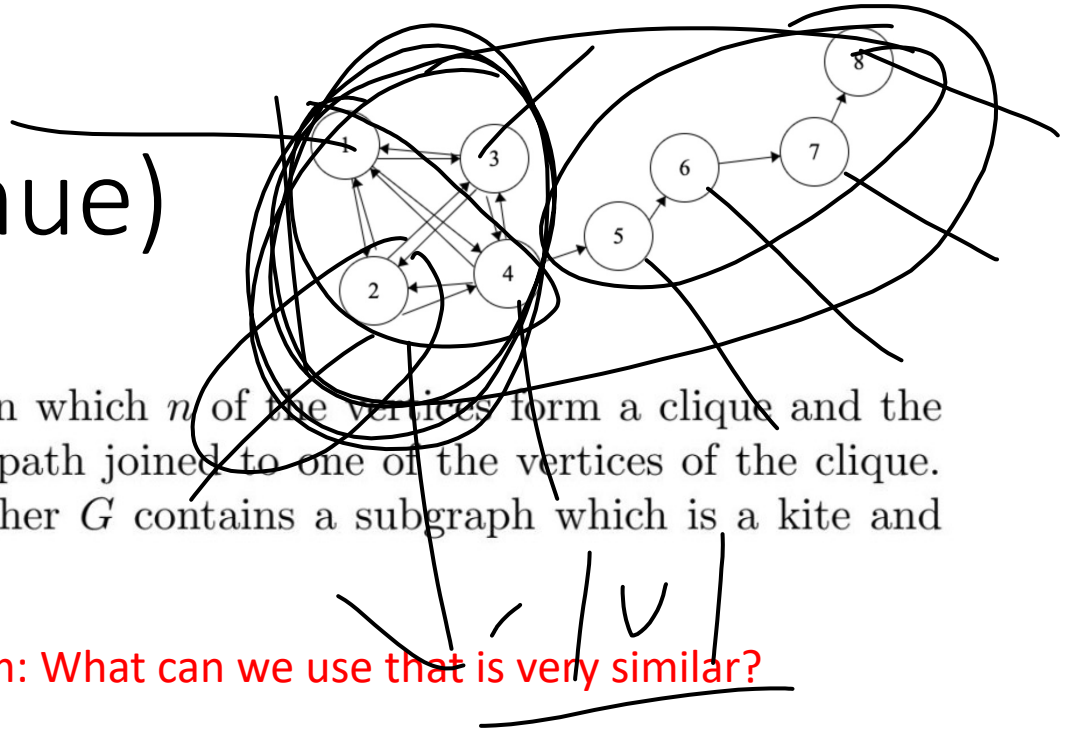
Example Problem (continue)



A kite is a graph on an even number of vertices, say $2n$, in which n of the vertices form a clique and the remaining n vertices are connected in a that consists of a path joined to one of the vertices of the clique. Given a graph and a goal g, the KITE problem asks whether G contains a subgraph which is a kite and which contains $2g$ nodes. Prove Kite is NP- complete.

1. Kite is NP: PROVIDED the inputs G and goal g can we VERIFY that our graph is a ”kite”?
   - verify that graph G is a kite by first looping through the g vertices that is one tail (check the edge set and make sure that no visited vertex has been visited before and is in a singular path) O(V+E)
   - verify that the remaining g vertices form a clique C by looping through (u, v) ∈ C (check the edge set to make sure that every vertex in click is connected to the other vertex in the clique) O(V+E)
   - both of the criteria above make this Kite problem NP because ***this verification can be done in polynomial time***

Example Problem (continue)



A kite is a graph on an even number of vertices, say $2n$, in which n of the vertices form a clique and the remaining n vertices are connected in a that consists of a path joined to one of the vertices of the clique. Given a graph and a goal g, the KITE problem asks whether G contains a subgraph which is a kite and which contains $2g$ nodes. Prove Kite is NP- complete.

1. Reduce a KNOWN NP-Complete problem to our Kite problem: What can we use that is very similar?
   - Reduce Clique to a Kite– How?
     - Let F be a graph that is a clique
     - add tails of size g to all v ∈ V to the Graph F that already contains a clique of size g (let this graph be called F')
     - verify whether or not we have a kite by using a KITE verification algorithm.
     - F' has a kite subgraph with 2g vertices if and only if there is a clique of size g in F
       - there is clique of size g in F so F' forms a 2g kite subgraph
       - F' exists and forms a 2g subgraph so F must have a clique of size g.
     - trace the kite that was returned and loop through the vertices and remove the tail (removing the tail will produce a clique)
       - if no kite was returned from the preprocessing step, then the algorithm will return no solution
     - Thus, we have reduced a KITE to a CLIQUE

Knapsack

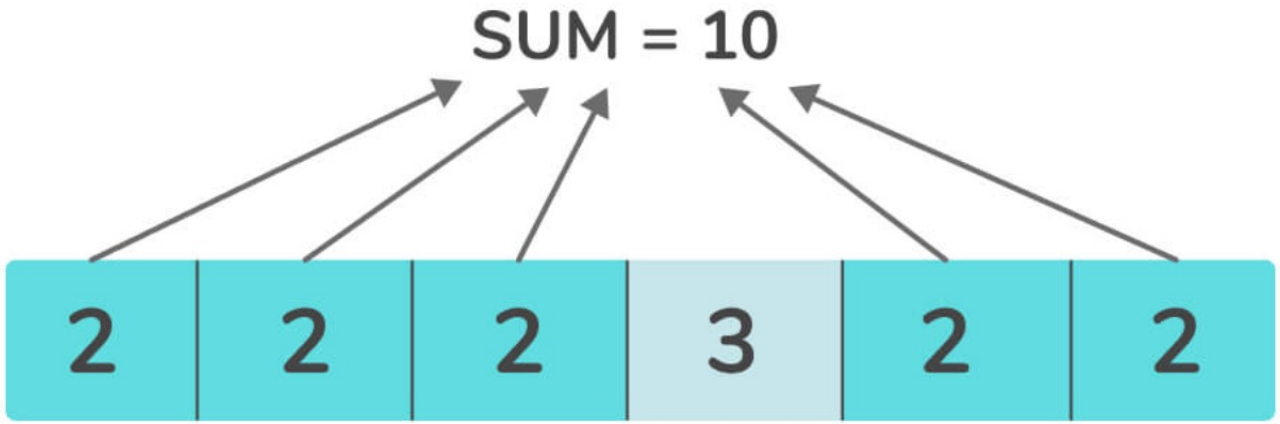
- Given weights and values of items, W and V, respectively, can a subset of items X {X1, X2, … Xn} be picked that satisfy the following constraints:

- Total Value of subset X >= Value

- Total Weight of subset X <= Weight

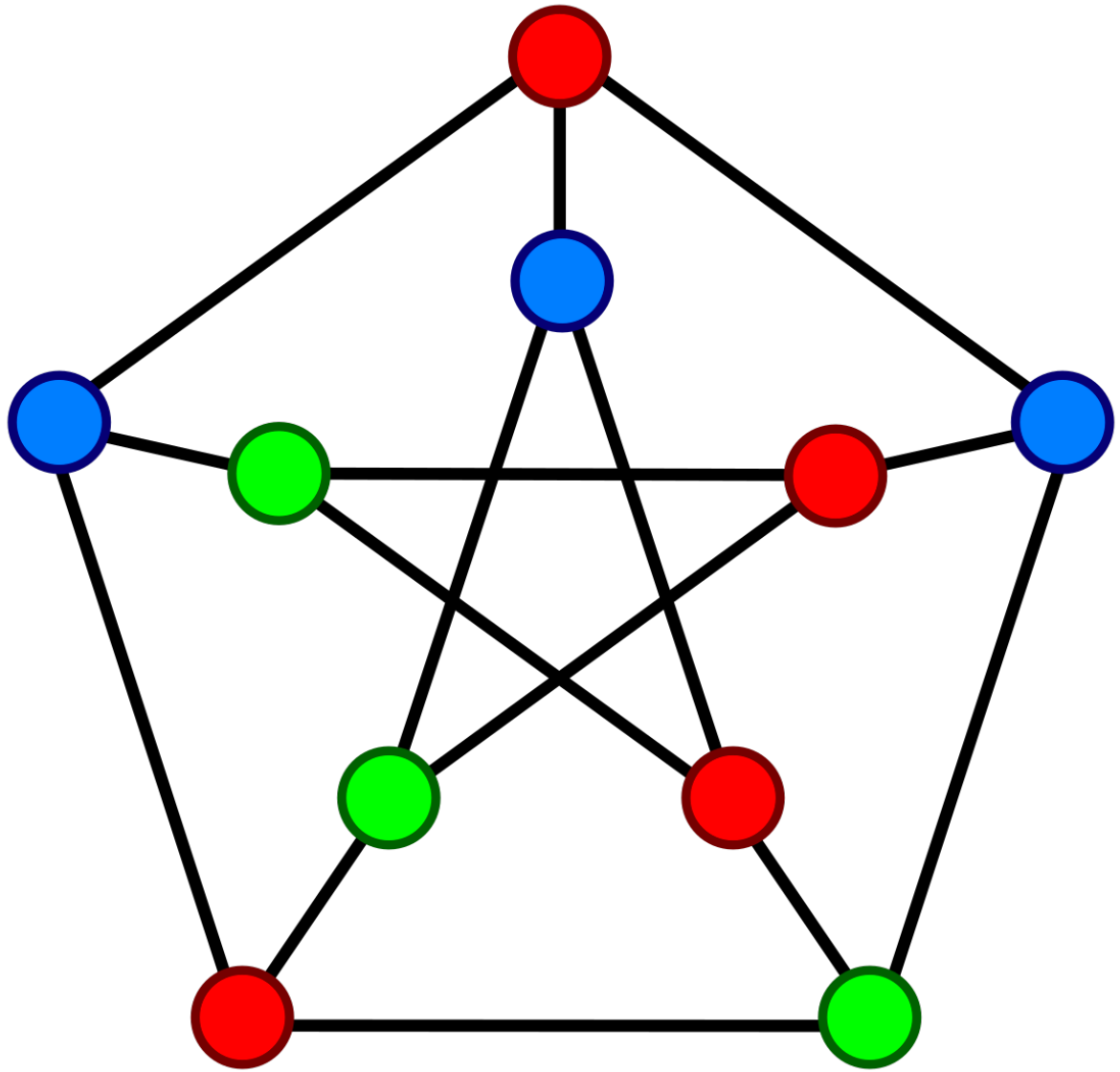
Subset Sum

- Subset Sum: Given a set S of integers and a target t, is there a subset S′ of S such that the sum of the elements of S′ is exactly t?



3-Coloring

- 3-Coloring Given a graph G(V, E), can we color the graph using **3 colors** in such a way that no two adjacent vertices share the same color?



3-Coloring Reduction Example

Problem 5 (*Optional, but highly recommended for the exam*)

Note: We won't grade this problem, but it's highly recommended that you solve the problem yourself.

Show that the following `4-coloring` problem is NP-Complete.

Input: An undirected graph $G = (V, E)$ and an integer $k = 4$.
Output: A 4-coloring of G if exists, else output NO

General Questions?

3. You've just ~~made a break through~~ and found a polynomial solution to the `LimitSAT`! The issue is that it's not exactly the same as normal ~~`SAT` because~~ each variable can only appear in at most three clauses, and there can only be at most three literals per clauses. ~~Can you show~~ that this is NP-Complete, thus showing P = NP (Obviously this is a made up scenario because we don't actually know if P = NP, but nonetheless, show that `LimitSAT` is in NP-Complete)?

4. As a student, you're trying to determine your schedule for the upcoming semester. For a particular day, there's some set of classes $s \in S$, each class has some meeting times (a class can meet multiple times in the day i.e. lecture and recitation). You need to determine whether you can schedule k non-overlapping classes throughout the day. Show this problem is NP-Complete.

Practice Exam #4

S_{org} = The Value of S_{org}

$S_{tot} = 2S_{org} - 2t \equiv 2t$

5. In `Daniel's-Two-Sum` problem, you're given a set of integers S, and you attempt to split the set up into two non-overlapping sets A and B such that $A \cup B = S$, $A \cap B = \emptyset$ and

$$\sum_{a \in A} a = \sum_{b \in B} b$$

$(S-t)$ $(S-t)$

Decide whether `Daniel's-Two-Sum` is NP-Complete and give sufficient evidence including either a proof it's NP-Complete or a polynomial time algorithm to solve it.

$(t) \neq (S_{org} - t) = S_{org}$

$\checkmark = S_{org} + 2t$

Practice Exam #5

6. Show that the `Subgraph-Isomorphism` is in the class NP-complete. The problem is as follows: given as input two undirected graphs G and H, determine whether G is a subgraph of H.

Practice Exam #6

7. The Frog Software Foundation has recently come out with Knapsack as a service, allowing you to solve the Knapsack decision problem, wherein you input your knapsack problem and a value, and the service will tell you if the best value in Knapsack is above the provided value. How could you use this to efficiently solve the Knapsack optimization problem wherein you're given the Knapsack parameters and you want to find the best possible value. Then discuss your algorithms runtime.

Practice Exam #7